Lean Proof Strategy Proposal for the Scheduling-Rules Document

Stuart Swan, Platform Architect, Siemens EDA

14 May 2026

0. Goal and scope

The goal is to formalize the document's **mathematical proof core** in Lean: trace compatibility, witness construction, cutpoint correspondence, elaboration, and one-way liveness/deadlock preservation. The Lean proof would certify the formalized theorem stack under the stated assumptions, not every informal prose statement in the document.

The document's own framing is conditional: the formal results depend on assumptions, observation-boundary choices, witness-selection conditions, and liveness/progress premises.

The Lean development should therefore be organized around explicit theorem instances and assumption bundles, rather than hidden global axioms.

---

1. Central comparison object

The proof should be parameterized by a selected comparison instance:

```
structure ComparisonInstance where
  Sys_ref    : System
  RTL_B      : System

  Sigma_ref  : Alphabet
  Sigma_post : Alphabet
  Sigma_DUT  : Alphabet

  env        : ReactiveEnvironment Sigma_post
  refChoice  : SysRefChoice

  combNetwork? : Option CombNetwork

  assumptions : AssumptionBundle
  witness     : WitnessBundle
  contracts   : ModelingContracts
```

Sys_ref is the chosen source reference model for the theorem instance:

- ordinary bounded-FIFO case: Sys_ref = Sys_B;
- elaborated case: Sys_ref = Sys_B^elab(E) for a well-formed elaboration package E.

This follows the updated Appendix Q direction: the formal theorems do not require a unique algorithm that constructs Sys_B^elab; instead, the chosen Sys_ref is part of the theorem instance.

---

2. Foundation layer: bucketed executions, not flat traces

The primary execution model should be **cycle-bucketed**, not a flat sequence of eps | tick | sigma e. The reason is that the document allows multiple same-cycle Σ-events without imposing artificial order among independent same-cycle events. The earlier review correctly noted that a flat trace would either impose a fake linearization or require an added bucket abstraction anyway.

Use:

```
structure State where
  cycle  : Nat
  epsPos : Nat
  store  : Store
```

with explicit advancement rules:

```
-- ε-steps advance epsPos only.
-- Same-cycle Σ-events advance epsPos only.
-- Clock ticks advance cycle and reset epsPos.
```

Define buckets:

```
structure CycleBucket (Σ : Type) where
  cycle       : Nat
  startState  : State
  epsPrefix   : List EpsStep
  sigmaEvents : Finset Σ
  sigmaOrder  : PartialOrderOn sigmaEvents
  epsSuffix   : List EpsStep
  endState    : State
  endQuiescent : Prop
```

```
def ExecutionBucketed (Σ : Type) :=
  List (CycleBucket Σ)

def InfiniteExecutionBucketed (Σ : Type) :=
  Nat → CycleBucket Σ
```

This provides the right foundation for:

- ε-quiescent cutpoints;
- rendezvous same-cycle commits;
- R2C combinational fixed-point buckets;
- J.1 ideal induction over unordered or partially ordered same-cycle units;
- B2/B3 temporal reasoning over infinite bucketed executions.

---

3. Events, operation instances, and source order

Define dynamic operation universes:

```
structure DynMsgInstance where
  proc      : Process
  channel   : Channel
  kind      : MsgKind     -- Push, Pop, PushNB, PopNB
  blocking  : Bool
  sourceLoc : SourceLoc
  instanceId : Nat

structure FrontierItem where
  proc      : Process
  itemKind  : FrontierItemKind
  sourceLoc : SourceLoc
  instanceId : Nat
```

Core source-order objects:

```
Qexec  : Process → Set DynMsgInstance
QOmega : Process → Set DynMsgInstance
Omega  : Process → Set FrontierItem
```

leP : Process → FrontierItem → FrontierItem → Prop
prefS : SyncCall → Set FrontierItem
suffS : SyncCall → Set FrontierItem

Non-blocking calls need explicit modeling:

- failed PushNB / PopNB polls are executed dynamic instances in Qexec, but produce no Σ-event;
- successful non-blocking calls produce the corresponding committed Push_c(v) / Pop_c(v) event;
- repeated NB calls are distinct dynamic source instances even if a request signal remains physically asserted.

This matches the document's issue/commit treatment of NB calls.

---

4. Reactive environment and modeling contracts

4.1 Reactive environment

"Fixed stimulus" should be modeled as a causal global strategy over Σ-history, not merely as a fixed stream of per-process inputs.

structure ReactiveEnvironment (Σ : Type) where
 inputAt : SigmaHistory Σ → ExternalInputs
 causal :
  ∀ h_1 h_2,
   SamePrefix h_1 h_2 →
   inputAt h_1 = inputAt h_2

This directly addresses the earlier issue that reactive stimulus must be interpreted as the same global Σ-causal behavior, not a function of local process history.

4.2 Modeling contracts

Direct-input contracts and array-access contracts should be parallel top-level modeling contracts, not partly folded into RRules and partly left separate.

structure ModelingContracts (C : ComparisonInstance) where
 directInputs : DirectInputContract C
 arrays : ArrayAccessMappingRules C

This keeps RRules close to the document's actual R-rule set and treats direct-input and external-array obligations as environment/modeling contracts.

---

5. Direct-input contract and late sampling

The direct-input contract should contain only primitive environmental stability/update obligations. The fact that late sampling preserves the logical anchor should be a **derived theorem**, not a separate hypothesis.

The document says hls_direct_input permits late physical sampling because the environment holds the signal stable after reset, while hls_direct_input_sync permits controlled updates only at the designated synchronization handshake. The document also states that logical signal Read/Write events are timestamped at the E2 anchor, not at any later physical RTL sampling cycle; late sampling is ε-internal.

Define:

```
structure DirectInputContract (C : ComparisonInstance) where
 stableAfterReset :
  ∀ sig t₁ t₂,
   DirectInput sig →
   AfterAllReceiversOutOfReset C sig t₁ →
   t₁ ≤ t₂ →
   InputValue C.env sig t₁ = InputValue C.env sig t₂

 syncControlledUpdateOnlyAtSync :
  ∀ sig s t,
   DirectInputSyncControlled sig s →
   ValueChangesAt C.env sig t →
   SyncHandshakeOccursAt C s t
```

Then prove:

```
theorem lateSamplingPreservesLogicalAnchor
 (C : ComparisonInstance)
 (hDI : DirectInputContract C)
 (sig : Signal)
 (anchor sampleT : Cycle)
 (hWindow : DirectInputAnchorWindow C sig anchor sampleT) :
```

```
  InputValue C.env sig anchor =
  InputValue C.env sig sampleT
```

Bucket-location rule:

```
-- The logical Read(sig,val) event belongs to the anchor bucket β_anchor.
-- A physical late sample, if modeled, is an ε-step in a later bucket.
-- The late ε-step creates no new Σ-event.
-- lateSamplingPreservesLogicalAnchor proves that the physical sample value
-- equals the logical anchored value.
```

This explicitly reconciles the bucket model with direct-input late sampling.

---

6. Temporal logic layer for B2/B3/B5/B6

B2 and B3 should not be opaque Prop fields. They are temporal assumptions over infinite executions.

The document defines B2 as weak fairness: if an action remains continuously enabled from some cycle onward, the scheduler must eventually select it. It applies to Push, Pop, and Sync operations. It also defines B3 as the P_push(c) / P_pop(c) channel-progress obligations.

Define temporal operators:

```
def AlwaysFrom
  (τ : InfiniteExecutionBucketed Σ)
  (t : Nat)
  (P : Nat → Prop) : Prop :=
  ∀ n, t ≤ n → P n


def EventuallyFrom
  (τ : InfiniteExecutionBucketed Σ)
  (t : Nat)
  (P : Nat → Prop) : Prop :=
  ∃ n, t ≤ n ∧ P n
```

Concrete fairness actions:

```
inductive FairAction
| push (q : DynMsgInstance)
```

```
| pop  (q : DynMsgInstance)
| sync (s : SyncCall)
```

Weak fairness:

```
def WeakFairness
 (C : ComparisonInstance)
 (τ : InfiniteExecutionBucketed C.Sigma_post) : Prop :=
 ∀ a t,
  AlwaysFrom τ t (fun n => FairActionEnabled C τ a n) →
  EventuallyFrom τ t (fun n => FairActionSelected C τ a n)
```

B3 channel-progress predicates should be defined in terms of the channel-semantics layer:

```
def PushStalledFull
 (C : ComparisonInstance)
 (τ : InfiniteExecutionBucketed C.Sigma_post)
 (c : Channel)
 (n : Nat) : Prop :=
 ∃ q,
  q.kind = MsgKind.Push ∧
  q.channel = c ∧
  Blocking q ∧
  BoundaryReqAt C τ q n ∧
  occAt C τ c n = capacity C c


def MatchingPopContinuouslyRequested
 (C : ComparisonInstance)
 (τ : InfiniteExecutionBucketed C.Sigma_post)
 (c : Channel)
 (n : Nat) : Prop :=
 ∃ q,
  q.kind = MsgKind.Pop ∧
  q.channel = c ∧
  Blocking q ∧
  BoundaryReqAt C τ q n


def PushProgressDischarged
 (C : ComparisonInstance)
```

```
 (τ : InfiniteExecutionBucketed C.Sigma_post)
 (c : Channel)
 (n : Nat) : Prop :=
 PopCommitOn C τ c n ∨ RendezvousCommitOn C τ c n
```

Similarly:

PopStalledEmpty
MatchingPushContinuouslyRequested
PopProgressDischarged

Then:

```
def P_push
 (C : ComparisonInstance)
 (τ : InfiniteExecutionBucketed C.Sigma_post)
 (c : Channel) : Prop :=
 ∀ t,
  AlwaysFrom τ t (fun n =>
   PushStalledFull C τ c n ∧
   MatchingPopContinuouslyRequested C τ c n) →
  EventuallyFrom τ t (fun n =>
   PushProgressDischarged C τ c n)


def P_pop
 (C : ComparisonInstance)
 (τ : InfiniteExecutionBucketed C.Sigma_post)
 (c : Channel) : Prop :=
 ∀ t,
  AlwaysFrom τ t (fun n =>
   PopStalledEmpty C τ c n ∧
   MatchingPushContinuouslyRequested C τ c n) →
  EventuallyFrom τ t (fun n =>
   PopProgressDischarged C τ c n)
```

This matches the document's conditional B3 reading: B3 forces progress only when both endpoints continuously request the matching transfer; a producer stalled on a full channel is not automatically a B3 violation if the consumer is not yet requesting the matching pop.

Bundle B-rules:

```
structure BRules (C : ComparisonInstance) where
 B1_determinism : DeterministicSigmaTrace C

 B2_weakFairness :
  ∀ τ, InfinitePostExecution C τ →
   WeakFairness C τ

 B3_channelProgress :
  ∀ τ c, InfinitePostExecution C τ →
   P_push C τ c ∧ P_pop C τ c

 B4_finiteEpsilonDepth : FiniteEpsilonDepth C
 B5_quiescentNormalization : QuiescentNormalization C
 B6_idleStutterTimeDivergence : IdleStutterTimeDivergence C
```

B2/B3 remain assumptions, but now they are structured temporal assumptions usable by K proofs.

---

7. Channel semantics and E4

Define channel state over the selected Sys_ref boundary:

```
capacity : C.Sys_ref.Channel → Nat
occ      : C.Sys_ref.Channel → State → Nat

BoundaryReq : DynMsgInstance → State → Prop
ChannelEnabled : DynMsgInstance → State → Prop
ChannelDisabled q σ := BoundaryReq q σ ∧ ¬ ChannelEnabled q σ
```

Capacity cases:

```
inductive CapacityKind
| rendezvous   -- B(c) = 0
| buffered     -- B(c) > 0
```

Expected lemmas:

```
BufferedPushEnabled
BufferedPopEnabled
RendezvousEnabled
FIFOOrderPreservation
```

NoDropNoDuplicate
OccupancyUpdatePush
OccupancyUpdatePop

E4 should be parameterized by Sys_ref, so in elaborated cases it ranges over explicit elaborated channels, not merely outer channels.

---

8. Appendix Q elaboration interface

Appendix Q should be introduced early. In the updated document, elaboration is not a late proof afterthought: in elaborated cases, Sys_ref = Sys_B^elab(E), and all references to B, occ, BoundaryReq, ChannelEnabled/Disabled, Front_P, and WFG are interpreted over the explicit channels of the elaborated model.

Chan_outer should be a parameter fixed by the chosen outer Sys_B, not a field chosen internally by the elaboration package.

```
structure ElaborationPackage
 (Sys_B : System)
 (Chan_outer : Type)
 (Σ_DUT : Type) where

 Sys_elab : System

 Chan_elab : Type
 finite_chan_elab : Fintype Chan_elab

 B_E   : Chan_elab → Nat
 occ_E : Chan_elab → State → Nat

 Pi_elab : Chan_elab → Option Chan_outer
 piSigma : Trace Sys_elab.Sigma → Trace Σ_DUT
 A_E     : Chan_outer → State → Nat

 outer_trace_preservation : Prop
 occupancy_preservation : Prop
 internal_refinement : Prop
 no_hidden_cross_channel_disablement : Prop
```

```
  wfg_visibility : Prop
  combinational_wellformedness : Prop
```

Reference choice:

```
inductive ReferenceKind
| ordinarySysB
| elaborated

structure SysRefChoice where
  outerSysB  : System
  Chan_outer : Type
  kind       : ReferenceKind
  elab?      : Option (ElaborationPackage outerSysB Chan_outer Σ_DUT)
  Sys_ref    : System
  sys_ref_matches_kind : Prop
```

For ordinarySysB, Sys_ref = outerSysB.

For elaborated, Sys_ref = E.Sys_elab.

This also handles sync-boundary epoch gates: if proof-relevant withholding exists, it must be represented by explicit gate channels in Sys_B^elab(E), not by hidden gating on an otherwise E4-enabled outer channel. The earlier review correctly flagged this as a point that must be explicit in the Lean strategy.

---

9. R-rules, E-rules, and modeling contracts

The R-rule bundle should include only the document's actual R-rules plus an optional Lean-side array-rule pointer if desired. It should **not** include a fictional R5_syncBoundaryNoCrossing; sync-boundary crossing is E5. The document explicitly labels "Messages cannot cross their immediate synchronization boundary" as E5.

```
structure RRules (C : ComparisonInstance) where
  R1_syncOrder : Prop
  R2_signalAnchorSequential : Prop
  R2C_combinationalFixedPoint : Prop
  R3_syncSemantics : Prop
  R4_messageIssueCommitOrder : Prop
```

E-rules:

```
structure ERules (C : ComparisonInstance) where
  E1_sync_order_preservation : Prop
  E2_signal_anchor_preservation : Prop
  E3_message_order_preservation : Prop
  E4_channel_capacity_semantics : Prop
  E5_sync_boundary_preservation : Prop
```

Array and direct-input conditions live in:

```
structure ModelingContracts (C : ComparisonInstance) where
  directInputs : DirectInputContract C
  arrays       : ArrayAccessMappingRules C
```

This avoids treating environment/design-side contracts inconsistently.

---

10. Array access mapping

External arrays are not merely internal Store. The document states that external arrays are mapped onto IO operations by an array access mapping layer, and that array accesses before a synchronization call must commit no later than the sync commit, while accesses after the sync must commit strictly after it. It also permits transformations such as caching, merging, splitting, and reordering when allowed by the mapping layer.

Define:

```
inductive MemoryEvent
| read  (array : ArrayId) (addr : Addr) (value : Value)
| write (array : ArrayId) (addr : Addr) (value : Value)

structure ArrayAccessMapping where
  sourceArrayAccess : SourceArrayAccess → List CoreIOEvent
  commitTime        : SourceArrayAccess → Cycle
  fixedLatency      : SourceArrayAccess → Nat

  conflictFreeReorderAllowed :
    SourceArrayAccess → SourceArrayAccess → Prop

  transformedEquivalent :
    SourceArrayAccess → List SourceArrayAccess → Prop
```

Rules:

```
structure ArrayAccessMappingRules (C : ComparisonInstance) where
 beforeSyncCommitsNoLater :
  ∀ a s,
   a ∈ prefS s →
   ArrayCommitTime a ≤ SyncCommitTime s

 afterSyncCommitsAfter :
  ∀ a s,
   a ∈ suffS s →
   SyncCommitTime s < ArrayCommitTime a

 issueConsistentWithFixedLatency :
  ∀ a,
   ArrayCommitTime a =
   ArrayIssueTime a + ArrayFixedLatency a

 transformedOpsRespectMapping :
  ∀ a transformed,
   TransformedFrom a transformed →
   MappingEquivalent a transformed

 conflictFreeReorderingOnly :
  ∀ a₁ a₂,
   Reordered a₁ a₂ →
   conflictFreeReorderAllowed a₁ a₂
```

For early milestones, it is acceptable to assume:

AllExternalArrayAccessesAlreadyMappedToCoreIO C

but the full proof strategy should include array mapping.

---

11. R2C combinational fixed point

ComparisonInstance now has:

combNetwork? : Option CombNetwork

R2C should use that field when combinational processes are in scope. The document says combinational Read/Write events occur in the observable bucket corresponding to the

cycle's ε-quiescent combinational fixed point, and that combinational signal IO is well formed only when the relevant network is deterministic and reaches a unique ε-fixed point.

Define:

```
structure CombNetwork where
 combStep : Store → Store → Prop
 acyclic  : Prop
 deterministic : Prop
```

acyclic and deterministic are proof obligations supplied by the network instance. They are not automatically checked unless the network representation is made concrete enough to prove them.

```
def CombQuiescent (N : CombNetwork) (σ : State) : Prop :=
 ¬ ∃ σ', N.combStep σ.store σ'.store


def CombFixedPointAtCycle
 (N : CombNetwork)
 (t : Cycle)
 (μ : Store) : Prop :=
 ReachesByCombSteps N t μ ∧
 IsFixedPoint N μ
```

Well-formedness:

```
structure R2C_WF (C : ComparisonInstance) where
 network_present :
  C.combNetwork?.isSome


 uniqueFixedPoint :
  ∀ t N,
   C.combNetwork? = some N →
   ∃! μ, CombFixedPointAtCycle N t μ


 sigmaReadsAtFixedPoint :
  ∀ e t μ N,
   C.combNetwork? = some N →
   CombReadWriteEvent e →
   e ∈ BucketSigmaEvents C t →
```

CombFixedPointAtCycle N t μ →
EventReadsWritesStore e μ

If combinational processes are out of scope for an early milestone, use:

SequentialOnly C

and postpone R2C_WF.

---

12. Issue/Commit well-formedness

Issue and Commit should be relational plus existence/uniqueness, not necessarily computable total functions.

IssueAt : Trace C.Sys_ref → DynMsgInstance → Cycle → Prop
CommitAt : Trace C.Sys_ref → DynMsgInstance → Cycle → Payload → Prop

Well-formedness:

structure IssueCommitWF (C : ComparisonInstance) where
issue_exists :
∀ τ q,
IssuedInTrace C τ q →
∃ t, IssueAt τ q t

issue_unique :
∀ τ q t_1 t_2,
IssueAt τ q t_1 →
IssueAt τ q t_2 →
$t_1 = t_2$

commit_exists_unique :
∀ τ q,
CommitsInTrace C τ q →
∃! tp : Cycle × Payload,
CommitAt τ q tp.1 tp.2

boundary_request_soundness : Prop
stable_attribution : Prop
back_to_back_issue : Prop

```
  sync_boundary_discipline : Prop
  nb_dynamic_instance_discipline : Prop
```

Derived timestamp:

```
noncomputable def IssueTime
  (hWF : IssueCommitWF C)
  (h : IssuedInTrace C τ q) : Cycle :=
  Classical.choose (hWF.issue_exists τ q h)
```

This preserves Appendix C's use of Issue(q) as an auxiliary timestamp while avoiding an unnecessary computability requirement.

---

13. Automatic flush and sync-boundary policy

Define:

```
Pending P σ
Front P σ
NextFront P σ
BlockingMsgNextFront P σ
Done P σ
Remain P σ
```

Automatic flush:

```
structure AutomaticFlush
  (C : ComparisonInstance)
  (P : Process)
  (τ : Trace C.RTL_B) where

  block_point : Prop
  no_new_later_work : Prop
  no_withdrawal : Prop
  frontier_minimality : Prop
  drain_only_downstream_progress : Prop
  sync_boundary_policy : SyncBoundaryPolicy C P τ
```

The sync-boundary policy must explicitly split the ordinary and elaborated cases:

```
inductive SyncBoundaryPolicy
  (C : ComparisonInstance)
```

```
  (P : Process)
  (τ : Trace C.RTL_B)

| no_withholding_in_ordinary_SysB :
    C.refChoice.kind = ReferenceKind.ordinarySysB →
    NoProofRelevantSyncBoundaryWithholding C P τ →
    SyncBoundaryPolicy C P τ

| explicit_gate_in_elab :
    ∀ E gateChannel,
      C.refChoice.kind = ReferenceKind.elaborated →
      GateChannelOf E gateChannel →
      OrdinaryChannelDisabledAtGate C P τ gateChannel →
      SyncBoundaryPolicy C P τ
```

In the ordinary Sys_B case, the sync-boundary clause does **not** add hidden gating. It asserts that no proof-relevant sync-boundary withholding is present. If such withholding is needed, the theorem instance must use Sys_B^elab(E).

---

14. Witness selector and Appendix L

With the assumed renumbering:

- Definition L.1 = witness selector / snooping;
- Lemma L.2 = snooping establishes WC;
- Corollary L.3 = NB-CF identity witness.

The uploaded document's witness selector is a partial strategy over ε-quiescent source cutpoints and ε-modeled choices whose outcomes may affect later Σ-visible behavior; it must be causally available from the matched RTL prefix and global Σ-causal history.

Avoid circularity by defining it over the partial construction so far:

```
structure WitnessSelector (C : ComparisonInstance) where
  choose :
    (τ_post_prefix : Prefix C.RTL_B) →
    (τ_ref_so_far : Prefix C.Sys_ref) →
    (cp : ChoicePoint τ_ref_so_far) →
    Option ChoiceOutcome
```

```
  causal :
    ∀ τp τr cp out,
      choose τp τr cp = some out →
      DependsOnlyOnAvailablePrefix τp τr cp out

  consistent_with_post_prefix :
    ∀ τp τr cp out,
      choose τp τr cp = some out →
      ConsistentWithRTLPrefix τp τr cp out
```

Witness compatibility:

```
def WC
  (C : ComparisonInstance)
  (W : WitnessSelector C) : Prop :=
  ∀ constructionPrefix,
    WitnessChoicesRespectMatchedPrefix C W constructionPrefix
```

Renamed Lemma L.2:

```
theorem L2_snooping_establishes_WC
  (C : ComparisonInstance)
  (W : WitnessSelector C)
  (hSnoop : SnoopingWrapperAssumptions C W) :
  WC C W
```

Renamed Corollary L.3:

```
theorem L3_nb_cf_identity_witness
  (C : ComparisonInstance)
  (P : Process)
  (hNBCF : NBCF C P) :
  WC C (IdentityOrArbitraryNBWitness C)
```

---

15. Concrete ObsSigma and deterministic replay

Do not define ObsSigma as an informal least fixed point. Define it as a concrete record of the source-state slices required for replay.

```
structure ObsSigma
  (C : ComparisonInstance)
  (W : WitnessSelector C)
  (P : Process)
  (σ : State) where

  control_pos : ControlPos P
  relevant_vars : RelevantVarSlice C P σ
  selected_eps_outcomes : SelectedOutcomeSlice C W P σ

  pending_slice : PendingSlice C P σ
  front_slice : FrontSlice C P σ
  nextfront_slice : NextFrontSlice C P σ

  boundary_req_slice : BoundaryReqSlice C P σ
  channel_fact_slice : ChannelFactSlice C P σ
  signal_anchor_slice : SignalAnchorSlice C P σ

  array_mapping_slice : ArrayMappingSlice C P σ
  direct_input_slice : DirectInputSlice C P σ
```

Then:

```
def SigmaRelevantEq C W P σ₁ σ₂ :=
  ObsSigma C W P σ₁ = ObsSigma C W P σ₂
```

Replay theorem:

```
theorem IDR_deterministic_replay
  (C : ComparisonInstance)
  (W : WitnessSelector C)
  (hB : BRules C)
  (hR : RRules C)
  (hE : ERules C)
  (hIC : IssueCommitWF C)
  (hContracts : ModelingContracts C)
  (hWC : WC C W)
  (P : Process)
  (σ₁ σ₂ : State)
```

(hObs : SigmaRelevantEq C W P σ_1 σ_2) :
SameNextSigmaRelevantBehavior C W P σ_1 σ_2

This same ObsSigma object should be reused by Lemma S1 and Corollary J.1.

---

16. Appendix I per-process construction

Bundle the assumptions:

```
structure IConstructionAssumptions
 (C : ComparisonInstance)
 (W : WitnessSelector C) where

 wellformed : C.WellFormed
 bRules : BRules C
 rRules : RRules C
 eRules : ERules C
 issueCommitWF : IssueCommitWF C
 implementation : ImplementationAssumptions C
 witnessCompatible : WC C W
 contracts : ModelingContracts C
```

Main theorems:

```
theorem I1_bounded_epsilon_closure
 (C : ComparisonInstance)
 (hB4 : C.assumptions.B.B4_finiteEpsilonDepth)
 (hB5 : C.assumptions.B.B5_quiescentNormalization) :
 ...

theorem I2_channel_progress_instance
 (C : ComparisonInstance)
 (hB3 : C.assumptions.B.B3_channelProgress) :
 ...

theorem I3_finite_extension_step
 (C : ComparisonInstance)
 (W : WitnessSelector C)
 (hI : IConstructionAssumptions C W) :
 ...
```

```
theorem I4_finite_prefix_construction
 (C : ComparisonInstance)
 (W : WitnessSelector C)
 (hI : IConstructionAssumptions C W) :
 ...

theorem I5_infinite_limit_construction
 (C : ComparisonInstance)
 (W : WitnessSelector C)
 (hI : IConstructionAssumptions C W) :
 ...
```

I3 should use IssueAt plus derived IssueTime, not an unconstrained guessed issue cycle.

---

17. Appendix J system-level correspondence

Define execution scopes:

```
def Exec_post (C : ComparisonInstance) (τ : Prefix C.RTL_B) : Prop :=
 ∃ τ∞,
  Extends τ∞ τ ∧
  InfinitePostExecution C τ∞ ∧
  FairProgressSatisfying C τ∞

def Exec_ref (C : ComparisonInstance) (τ : Prefix C.Sys_ref) : Prop :=
 ∃ τ∞,
  Extends τ∞ τ ∧
  RefAdmissible C τ∞
```

Directed compatibility:

```
CompatibleP   : ComparisonInstance → Process → Prefix C.Sys_ref → Prefix C.RTL_B → Prop
CompatibleSys : ComparisonInstance → Prefix C.Sys_ref → Prefix C.RTL_B → Prop
```

Do not make ≈ into a symmetric equivalence relation. The main direction is post-to-reference; the reverse is restricted to RTL-consistent source prefixes.

J.0:

```
theorem J0_pairwise_composition
  (C : ComparisonInstance)
  (hC : C.WellFormed)
  (hB : BRules C)
  (hR : RRules C)
  (hE : ERules C)
  (hIC : IssueCommitWF C)
  (hContracts : ModelingContracts C)
  (τ_ref : Prefix C.Sys_ref)
  (τ_post : Prefix C.RTL_B)
  (hProc : ∀ P, CompatibleP C P τ_ref τ_post)
  (hChan : ChannelsWellFormedAndMatched C τ_ref τ_post) :
  CompatibleSys C τ_ref τ_post
```

J.1:

```
structure JAssumptions
  (C : ComparisonInstance)
  (W : WitnessSelector C) where

  wellformed : C.WellFormed
  bRules : BRules C
  rRules : RRules C
  eRules : ERules C
  issueCommitWF : IssueCommitWF C
  implementation : ImplementationAssumptions C
  witnessCompatible : WC C W
  contracts : ModelingContracts C
  environment : ReactiveEnvironmentWF C.env

theorem J1_post_to_ref
  (C : ComparisonInstance)
  (W : WitnessSelector C)
  (hJ : JAssumptions C W)
  (τ_post : Prefix C.RTL_B)
  (hExec : Exec_post C τ_post) :
  ∃ τ_ref,
    Exec_ref C τ_ref ∧
    CompatibleSys C τ_ref τ_post
```

Restricted reverse:

```
theorem J1_ref_to_post_restricted
  (C : ComparisonInstance)
  (W : WitnessSelector C)
  (hJ : JAssumptions C W)
  (τ_ref : Prefix C.Sys_ref)
  (hExec : Exec_ref C τ_ref)
  (hConsistent : RTLConsistentSourcePrefix C W τ_ref) :
  ∃ τ_post,
    Exec_post C τ_post ∧
    CompatibleSys C τ_ref τ_post
```

Cutpoint correspondence:

```
structure CutpointCorrespondence
  (C : ComparisonInstance)
  (σ_ref : State)
  (σ_post : State) where

  nextfront_agreement : Prop
  frontier_guard_value_boundaryreq_agreement : Prop
  buffered_channel_state_agreement : Prop
  raw_rendezvous_endpoint_request_agreement : Prop
  pending_blocking_enablement_agreement : Prop

  direct_input_anchor_agreement : Prop
  array_mapping_agreement : Prop
  r2c_fixed_point_agreement : Prop
```

```
theorem J1_cutpoint_correspondence
  (C : ComparisonInstance)
  (W : WitnessSelector C)
  (hJ : JAssumptions C W)
  (τ_ref : Prefix C.Sys_ref)
  (τ_post : Prefix C.RTL_B)
  (hCompat : CompatibleSys C τ_ref τ_post)
  (hNorm : EpsilonNormalizedPair C τ_ref τ_post) :
  ∃ σ_ref σ_post,
    TerminalCutpoint τ_ref σ_ref ∧
```

TerminalCutpoint τ_post σ_post ∧
CutpointCorrespondence C σ_ref σ_post

---

18. Appendix K WFG and liveness

Define WFG over selected Sys_ref boundaries:

```
def WFG
 (C : ComparisonInstance)
 (side : Side)
 (σ : State) : Digraph Process :=
 ...

def WFGEdge C side σ P Q :=
 ∃ q,
  q ∈ Front C P σ ∧
  ChannelDisabled C q σ ∧
  ComplementOwner C q = Q ∧
  InternalChannel C q.channel
```

Edges are generated from Front_P(σ), not arbitrary non-frontier asserted requests.

K assumptions should preserve the document's A1–A11 as a bundle, and add Lean-side auxiliary contracts without pretending they are doc-numbered assumptions.

```
structure DocKAssumptions
 (C : ComparisonInstance)
 (W : WitnessSelector C) where

 A1_point_to_point_channels : Prop
 A2_chosen_Sys_ref_boundaries : Prop
 A3_R_E_rule_conformance : RRules C ∧ ERules C
 A4_B_rules : BRules C
 A5_source_liveness : InternalMPDeadlockFree C.Sys_ref
 A6_issue_commit_wf : IssueCommitWF C
 A7_witness_bundle : WC C W
 A8_epsilon_normalization : Prop
 A9_single_transfer_per_cycle : Prop
```

A10_reset_empty_or_initial_accounting : Prop
A11_Kepsilon_WFG_inert : Prop

Lean-side additions:

```
structure KAssumptions
 (C : ComparisonInstance)
 (W : WitnessSelector C) where

 doc : DocKAssumptions C W
 contracts : ModelingContracts C
 r2c? : Option (R2C_WF C)
```

This avoids fictional A12/A13 labels.

K lemmas:

```
theorem K0_frontier_blocking_classification
 (C : ComparisonInstance)
 (W : WitnessSelector C)
 (hK : KAssumptions C W)
 (σ : State)
 (hQ : EpsQuiescent σ) :
 ...

theorem K0a_rendezvous_endpoint_request_agreement
 (C : ComparisonInstance)
 (W : WitnessSelector C)
 (hCorr : CutpointCorrespondence C σ_ref σ_post)
 (c : Channel)
 (hB0 : capacity c = 0) :
 Req_prod c σ_ref = Req_prod c σ_post ∧
 Req_cons c σ_ref = Req_cons c σ_post

theorem K1_deadlock_characterization
 (C : ComparisonInstance)
 (W : WitnessSelector C)
 (hK : KAssumptions C W)
 (σ : State)
 (hReach : Reachable C.RTL_B σ)
```

```
 (hQ : EpsQuiescent σ) :
 InternalMPDeadlockAt C σ ↔
 ExistsClosedDrainClosedWFGCycle C σ

theorem K2_dependency_refinement
 (C : ComparisonInstance)
 (W : WitnessSelector C)
 (hK : KAssumptions C W)
 (σ_ref σ_post : State)
 (hCorr : CutpointCorrespondence C σ_ref σ_post) :
 WFG C .ref σ_ref = WFG C .post σ_post

theorem K2a_drain_closure_transfer
 (C : ComparisonInstance)
 (W : WitnessSelector C)
 (hK : KAssumptions C W)
 (D : Finset Process)
 (σ_ref σ_post : State)
 (hCorr : CutpointCorrespondence C σ_ref σ_post)
 (hDrainPost : DrainClosed C .post D σ_post) :
 DrainClosed C .ref D σ_ref
```

Exec-post membership at a deadlock cutpoint should be explicit:

```
theorem deadlock_cutpoint_in_Exec_post
 (C : ComparisonInstance)
 (W : WitnessSelector C)
 (hK : KAssumptions C W)
 (σ_post : State)
 (hReach : Reachable C.RTL_B σ_post)
 (hDead : ExistsClosedDrainClosedWFGCycle C σ_post)
 (hIdle : C.assumptions.B.B6_idleStutterTimeDivergence) :
 ∃ τ_post,
  TerminalCutpoint τ_post σ_post ∧
  Exec_post C τ_post
```

Main K theorem:

```
theorem K1_liveness_preservation
 (C : ComparisonInstance)
```

(W : WitnessSelector C)
(hK : KAssumptions C W) :
InternalMPDeadlockFree C.Sys_ref →
InternalMPDeadlockFree C.RTL_B

This is intentionally one-way. Do not state:

InternalMPDeadlockFree C.Sys_ref ↔ InternalMPDeadlockFree C.RTL_B

---

19. Appendix R facade

Appendix R should be formalized last as aliases/wrappers over earlier theorems.

Use Lean names that avoid the doc's R.2 collision:

theorem R_lemma1_front_nextfront_classification := ...
theorem R_theorem1_prefix_matching := ...
theorem R_corollary1_cutpoint_correspondence := ...
theorem R_corollary1a_rendezvous_endpoint_request_agreement := ...
theorem R_lemma2_dependency_refinement := K2_dependency_refinement
theorem R_theorem2_closed_cycle_and_one_way_drain_transfer := ...

Do not call cutpoint correspondence R1_cutpoint_correspondence; in the doc structure, it corresponds to a corollary, not Theorem R.1.

---

20. What remains as hypotheses

These remain theorem parameters or design/tool obligations:

1. R-rules R1, R2, R2C, R3, R4.
2. E-rules E1–E5.
3. B1–B6, with B2/B3 encoded temporally.
4. Implementation assumptions such as I.A0.
5. Witness compatibility WC, derived from Lemma L.2 or Corollary L.3 where applicable.
6. Direct-input contracts.
7. Array access mapping rules.

8. Elaboration package E, when Sys_ref = Sys_B^elab(E).
9. Source-level internal message-passing deadlock freedom for K.

---

21. What should be proved inside Lean

Lean should prove:

- bucketed execution infrastructure;
- temporal lemmas for AlwaysFrom / EventuallyFrom;
- channel-capacity lemmas for rendezvous and buffered cases;
- issue existence/uniqueness and derived IssueTime;
- direct-input late-sampling preservation from primitive environment contracts;
- array sync-boundary commit-order preservation;
- R2C fixed-point observation lemmas from R2C_WF;
- automatic-flush frontier lemmas;
- Lemma L.2: snooping establishes WC;
- Corollary L.3: NB-CF identity witness;
- I.DR over concrete ObsSigma;
- Appendix I finite and infinite witness construction;
- Proposition J.0;
- Theorem J.1 post-to-reference;
- restricted reverse theorem for RTL-consistent source prefixes;
- Corollary J.1 cutpoint correspondence;
- K.0/K.0a/K.1/K.2/K.2a;
- deadlock cutpoint membership in Exec_post;
- Theorem K.1 one-way liveness preservation;
- Appendix R aliases.

---

22. Development milestones

Milestone A — Sequential, blocking-only, non-elaborated core

Scope:

- Sys_ref = Sys_B;
- sequential processes only;
- no NB calls;
- no snooping;
- no external arrays;
- no direct-input late sampling;
- no combinational R2C;
- bucketed execution model.

Targets:

J1_post_to_ref_blocking
J1_cutpoint_correspondence_blocking
K1_liveness_preservation_blocking

Milestone B — Temporal B2/B3

Add:

- infinite bucketed executions;
- AlwaysFrom, EventuallyFrom;
- concrete FairAction;
- P_push, P_pop.

Target:

K1_liveness_preservation_with_temporal_progress

Milestone C — Non-blocking under NB-CF

Add:

- Qexec;
- failed NB polls as ε;

- successful NB transfers as Σ;
- Corollary L.3.

Target:

J1_post_to_ref_nb_cf

Milestone D — Witness selector / snooping

Add:

- partial-prefix WitnessSelector;
- Lemma L.2.

Target:

J1_post_to_ref_with_snooping

Milestone E — Direct inputs

Add:

- DirectInputContract;
- derived lateSamplingPreservesLogicalAnchor;
- bucket placement for logical anchor vs physical late sample.

Target:

J1_post_to_ref_direct_inputs

Milestone F — External arrays

Add:

- MemoryEvent;
- ArrayAccessMapping;
- sync-relative array commit rules.

Target:

J1_post_to_ref_external_arrays

Milestone G — R2C

Add:

- combNetwork?;
- CombNetwork;
- R2C_WF;
- fixed-point bucket observation semantics.

Target:

J1_post_to_ref_with_R2C

Milestone H — Elaboration

Add:

- ElaborationPackage Sys_B Chan_outer Σ_DUT;
- Π_elab;
- π_Σ,E;
- A_E;
- explicit-channel interpretation of B, occ, BoundaryReq, ChannelEnabled, Front, and WFG.

Targets:

J1_post_to_ref_elab
K1_liveness_preservation_elab

Milestone I — Appendix R facade

Add compact R theorem wrappers after the full proof stack is available.

---

23. Expected difficulty

The hardest pieces remain:

1. Bucketed execution and ideal induction.
2. Temporal B2/B3 over infinite executions.
3. Concrete ObsSigma and I.DR.
4. Appendix J finite-ideal construction.
5. Direct-input late-sampling contracts.

6. External array access mapping.
7. R2C fixed-point semantics.
8. Appendix Q elaboration obligations.
9. Appendix K Exec_post membership at deadlock cutpoints.

A polished full formalization is plausibly in the **10k–18k Lean line** range if direct inputs, arrays, R2C, temporal fairness/progress, and elaboration are all included. A core sequential/blocking/non-elaborated proof would be substantially smaller.